In [7]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

import xgboost as xgb
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.model_selection import train_test_split
from sklearn.inspection import permutation_importance
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
# import shap

import warnings
warnings.filterwarnings("ignore")
```

In [8]:
```
data=pd.read_csv('Churn_Modelling.csv')
```

- Customer ID: A unique identifier for each customer
- 客户 ID：每个客户的唯一标识符
- Surname: The customer's surname or last name
- 姓氏：客户的姓氏或姓氏
- Credit Score: A numerical value representing the customer's credit score
- 信用评分：代表客户信用状况的数值
- Geography: The country where the customer resides
- 地理位置：客户所在的国家/地区
- Gender: The customer's gender
- 性别：顾客的性别
- Age: The customer's age. 年龄：顾客的年龄。
- Tenure: The number of years the customer has been with the bank
- 服务年限：客户在银行的账户持有年限。
- Balance: The customer's account balance
- 余额：客户的账户余额
- NumOfProducts: The number of bank products the customer uses (e.g., savings account, credit card)
- 产品数量：客户使用的银行产品数量（例如，储蓄账户、信用卡）
- HasCrCard: Whether the customer has a credit card
- HasCrCard：客户是否持有信用卡
- IsActiveMember: Whether the customer is an active member
- IsActiveMember：客户是否为活跃会员
- EstimatedSalary: The estimated salary of the customer
- 预估薪资：客户的预估薪资
- Exited: Whether the customer has churned (Target Variable)
- 已退出：客户是否已流失（目标变量）

In [11]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   RowNumber        10000 non-null  int64  
 1   CustomerId       10000 non-null  int64  
 2   Surname          10000 non-null  object 
 3   CreditScore      10000 non-null  int64  
 4   Geography        10000 non-null  object 
 5   Gender           10000 non-null  object 
 6   Age              10000 non-null  int64  
 7   Tenure           10000 non-null  int64  
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64  
 10  HasCrCard        10000 non-null  int64  
 11  IsActiveMember   10000 non-null  int64  
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64  
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

In [14]:
```
data.head(10)
```

Out[14]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Ter
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	
5	6	15574012	Chu	645	Spain	Male	44	
6	7	15592531	Bartlett	822	France	Male	50	
7	8	15656148	Obinna	376	Germany	Female	29	
8	9	15792365	He	501	France	Male	44	
9	10	15592389	H?	684	France	Male	27	

In [15]:
```
data['Geography'].unique()
```

Out[15]:
```
array(['France', 'Spain', 'Germany'], dtype=object)
```

In [17]:
```
data['Gender'].unique()
```

Out[17]:
```
array(['Female', 'Male'], dtype=object)
```

In [18]:
```
data.describe()
```

Out[18]:

	RowNumber	CustomerId	CreditScore	Age	Tenure	
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250

In [19]:
```
data.drop(columns=['RowNumber','CustomerId','Surname'],inplace=True)
data
```

Out[19]:

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts
0	619	France	Female	42	2	0.00	1
1	608	Spain	Female	41	1	83807.86	1
2	502	France	Female	42	8	159660.80	3
3	699	France	Female	39	1	0.00	2
4	850	Spain	Female	43	2	125510.82	1
...	...	...	...	...	...	...	...
9995	771	France	Male	39	5	0.00	2
9996	516	France	Male	35	10	57369.61	1
9997	709	France	Female	36	7	0.00	1
9998	772	Germany	Male	42	3	75075.31	2
9999	792	France	Female	28	4	130142.79	1

10000 rows × 11 columns

In [20]:
```
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_mat
```

In [21]:
```
labelencoder=LabelEncoder()
data['Gender']=labelencoder.fit_transform(data['Gender'])
data['Geography']=labelencoder.fit_transform(data['Geography'])
data
```

Out[21]:

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts
0	619	0	0	42	2	0.00	1
1	608	2	0	41	1	83807.86	1
2	502	0	0	42	8	159660.80	3
3	699	0	0	39	1	0.00	2
4	850	2	0	43	2	125510.82	1
...	...	...	...	...	...	...	...
9995	771	0	1	39	5	0.00	2
9996	516	0	1	35	10	57369.61	1
9997	709	0	0	36	7	0.00	1
9998	772	1	1	42	3	75075.31	2
9999	792	0	0	28	4	130142.79	1

10000 rows × 11 columns

In [23]:
```
data.dtypes
```

Out[23]:
```
CreditScore          int64
Geography            int64
Gender               int64
Age                  int64
Tenure               int64
Balance            float64
NumOfProducts        int64
HasCrCard            int64
IsActiveMember       int64
EstimatedSalary    float64
Exited               int64
dtype: object
```

In [24]:
```
x=data.drop(columns='Exited') # droping coulmn from the feature
y=data['Exited']
```

In [75]:
```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,shuffle=True,r
```

In [76]:
```
model=RandomForestClassifier()
model.fit(x_train,y_train)
```

Out[76]:

▾ RandomForestClassifier

RandomForestClassifier()

In [77]:
```
model.score(x_train,y_train)
```

Out[77]:
```
0.9997333333333334
```

In [78]:
```
y_pred = model.predict(x_test)
y_pred
```

Out[78]:
```
array([0, 1, 0, ..., 0, 0, 1])
```

In [79]:
```
len(y_pred)
```

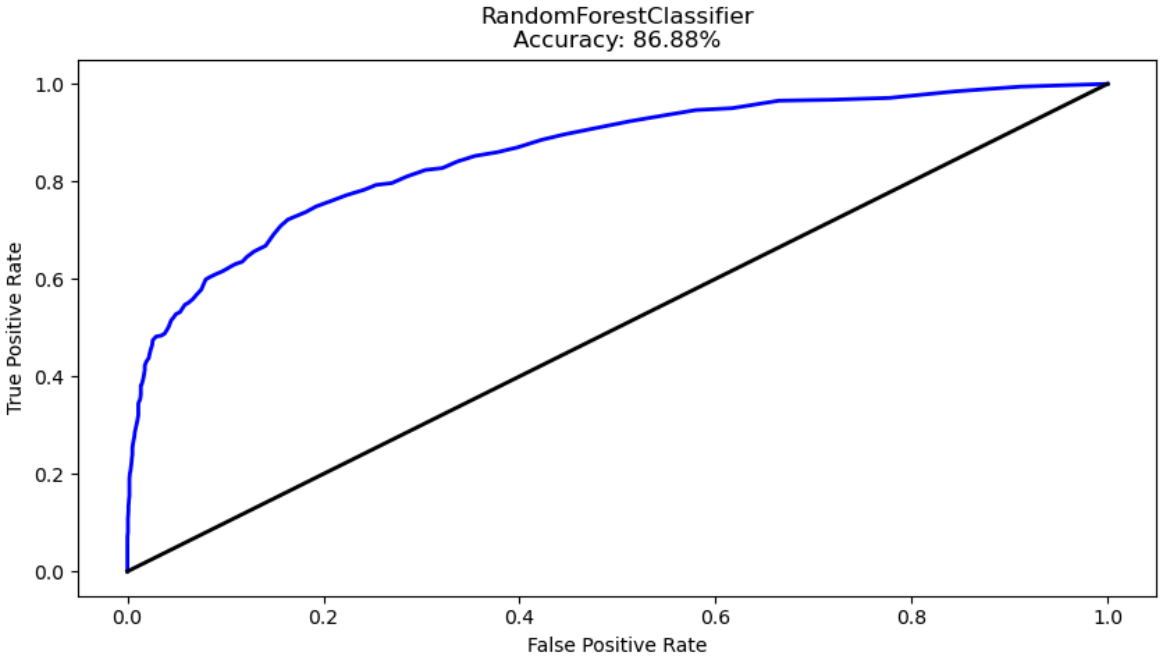
Out[79]:
```
2500
```

In [80]:
```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: {:.2f}%".format(accuracy * 100))
```

```
Accuracy: 86.88%
```

In [81]:
```
# visualization
y_prob = model.predict_proba(x_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure(figsize=(10, 5))
plt.plot(fpr, tpr, color='blue', lw=2)
plt.plot([0, 1], [0, 1], color='black', lw=2)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('RandomForestClassifier\nAccuracy: {:.2f}%'.format(accuracy * 100))
plt.show()
```



In [ ]:
```

```